root@seclab:~# ./acceso_control.sh --start

CLASE 6

FALLAS EN CONTROL DE ACCESO & IDOR

> Autenticación · Sesiones · IDOR · Escalada de privilegios

[STATUS] target acquired · exploit lab activo

root@seclab:~/clase6$ cat objetivo.txt && ls modulos/

> Objetivo de la misión

Evaluar la lógica de autenticación y los mecanismos que autorizan a los usuarios a interactuar con los recursos del backend.

01

Broken Authentication

Fuerza bruta en logins + enumeración de usuarios válidos

02

Gestión de Sesiones

Tokens, cookies débiles y ataques a la sesión

03

IDOR

Referencias directas inseguras a objetos del backend

04

Escalada de Privilegios

Horizontal (mismo nivel) y vertical (hacia admin)

// MÓDULO

01

BROKEN AUTHENTICATION

> Gestión de autenticación ineficaz

```
root@seclab:~/clase6$ hydra -L users.txt -P rockyou.txt target
```

> Ataques de fuerza bruta en logins

VECTORES

- Fuerza bruta clásica: probar todo el keyspace
- Diccionario: rockyou, listas de credenciales
- Credential stuffing: pares user:pass filtrados
- Password spraying: 1 pass contra N usuarios (evade lockout)

DEFENSAS

- Rate limiting + lockout progresivo por IP/cuenta
- MFA / 2FA resistente a phishing
- CAPTCHA tras N intentos fallidos
- Hashing fuerte: bcrypt / argon2 + salt único
- Monitoreo y alertas de logins anómalos

root@seclab:~/clase6$ diff respuesta_user_valido respuesta_user_invalido

> Enumeración de usuarios válidos

Señales que delatan si un usuario existe, incluso sin credenciales correctas.

MENSAJE	"Usuario no existe" vs "Password incorrecta"
TIMING	La respuesta tarda distinto si el user es válido
HTTP	Status / headers / longitud de respuesta diferentes
REGISTRO	"Ese email ya está registrado" en el signup
RESET	Reset de password confirma si la cuenta existe

[FIX] Respuestas y tiempos uniformes para user válido e inválido.

// MÓDULO

02

GESTIÓN DE SESIONES

> Tokens, cookies débiles y ataques a la sesión

root@seclab:~/clase6$ inspect Set-Cookie; decode session.token

> Ataques a la gestión de sesiones

▸ Session Hijacking

Robo del ID de sesión (XSS, sniffing) para suplantar al usuario

▸ Session Fixation

Forzar un session ID conocido antes del login

▸ Token predecible

IDs secuenciales o con poca entropía → adivinables

▸ Cookie sin flags

Falta de HttpOnly / Secure / SameSite expone la cookie

▸ JWT mal validado

alg:none, firma no verificada, secreto débil

▸ Sin expiración

La sesión nunca caduca ni se invalida al logout

root@seclab:~/clase6$ harden Set-Cookie

> Cookie débil vs. cookie endurecida

[VULNERABLE]

```
Set-Cookie: SID=1024;
  path=/
```

- ID secuencial y predecible
- Sin HttpOnly → accesible por JS (XSS)
- Sin Secure → viaja en HTTP plano
- Sin SameSite → expuesta a CSRF
- Sin expiración ni rotación

[ENDURECIDA]

```
Set-Cookie: SID=<rand-256b>;
  HttpOnly; Secure;
  SameSite=Strict
```

- Alta entropía (CSPRNG)
- HttpOnly bloquea acceso por JS
- Secure obliga HTTPS
- SameSite mitiga CSRF
- Expira y rota tras el login

// MÓDULO

03

IDOR

> Insecure Direct Object References

root@seclab:~/clase6$ GET /api/invoice?id=1337

> IDOR: análisis conceptual

La app expone un identificador del objeto y confía en él sin verificar que el usuario tenga permiso sobre ese recurso.

DEMO — alteración de parámetro

GET /account/**1024**/details → 200 OK (mi cuenta)

GET /account/**1025**/details → **200 OK (cuenta AJENA · leak!)**

Parámetros

?id= · /user/42 · ?file=report.pdf

IDs secuenciales

Enteros fáciles de iterar (1,2,3…)

Cuerpos / cookies

Campos ocultos, JSON, headers manipulables

root@seclab:~/clase6$ enforce authorization per-object

> Cómo cerrar un IDOR

✓	**Autorización por objeto**	Verificar SIEMPRE que el recurso pertenece al usuario autenticado, no solo que está logueado
✓	**IDs no adivinables**	UUID v4 / identificadores opacos en lugar de enteros secuenciales
✓	**Referencias indirectas**	Mapear IDs internos a referencias por-sesión del lado servidor
✓	**Control centralizado**	Un único punto de chequeo de permisos, no disperso por endpoint
✓	**Deny by default**	Negar por defecto; registrar y alertar accesos denegados

// MÓDULO

04

ESCALADA DE PRIVILEGIOS

> Horizontal y vertical

root@seclab:~/clase6$ whoami && sudo -l

> Horizontal vs. Vertical

◀▶ HORIZONTAL

Acceso a recursos de otra cuenta del MISMO nivel de privilegio.

userA ⟶ datos de userB

- Ver pedidos/facturas ajenos
- Editar el perfil de otro usuario
- A menudo nace de un IDOR

▲ VERTICAL

Obtener un ROL superior: de usuario común a administrador.

user ⟶ ADMIN / root

- Forzar endpoints /admin
- Alterar role=user → role=admin
- Function-level access control roto

root@seclab:~/clase6$ run access-control-testing.sh

> Metodología de testeo

01 **Mapear roles y recursos**

Identificar cuentas, roles y objetos protegidos

02 **Crear 2 sesiones**

userA, userB y si se puede un admin de control

03 **Interceptar tráfico**

Proxy (Burp/ZAP) para capturar IDs y tokens

04 **Alterar identificadores**

Cambiar id/params y repetir con otra sesión

05 **Probar endpoints de rol**

Acceder a funciones admin desde cuenta común

06 **Documentar impacto**

Evidencia, severidad y remediación

root@seclab:~# echo $CONCLUSION

La autorización se verifica en el servidor. Siempre.

- Autenticar ≠ autorizar: validá permisos en cada objeto y función
- Endurecé sesiones: tokens con entropía, HttpOnly/Secure/SameSite
- IDs opacos + deny-by-default cierran la mayoría de los IDOR

> fin de la clase 6 _